The Pi-Eye

– Text Reader and Picture Analyzer for Visually Impaired

A Project report submitted to

St. Albert's College (Autonomous), Ernakulam in partial

Fulfillment of the requirement for the award of the degree of

MASTER OF SCIENCE IN

PHYSICS

By

AMOS RAM REHUM

Reg No: 1602190001

DEPARTMENT OF PHYSICS

ST. ALBERT'S COLLEGE (AUTONOMOUS)

ERNAKULAM

DEPARTMENT OF PHYSICS (AUTONOMOUS)

ST. ALBERT'S COLLEGE

ERNAKULAM

CERTIFICATE

This is to certify that this is a bonafide record of the project work entitled ***The Pi-Eye – Text Reader and Picture Analyzer for Visually Impaired,*** *done by* ***Amos Ram Rehum*** *of M.Sc. Physics, St. Albert's College during the year 2016-2018.*

Signature of examiner

Reg. No: 1602190001

Date: 19.04.2018

Lecturer in charge

ACKNOWLEDGEMENT

First and foremost, I would like to thank the Almighty for all the blessing bestowed.

I extent my sincere thanks to **Prof. Shaji Joseph** whose support and guidance was almost valuable during the course of this work.

Last but not the least I take this opportunity to thank all our teachers, especially **Prof. Lawrel Gregory**, Head of the Department of Physics, St. Albert's college, our dear friends and parents for all the invaluable moral support and help.

Amos Ram Rehum

DECLARATION

I hereby declare that the work presented in the dissertation entitled "The Pi-Eye – Text Reader and Picture Analyser for Visually Impaired" is done by me under the guidance of Prof. Shaji Joseph, Associate Professor in Department of Physics, St. Albert's College, Ernakulam, during the year 2016 – 2018 and the project is a fresh work with no part of it has been included in any other dissertation submitted previously for the award of any other degree.

19/04/18

Amos Ram Rehum

ABSTRACT

There are so many persons who find difficulty in vision. They are always exempted from the mainstream in almost all fields. Especially, visually impaired students find much difficult to withstand in the fast-moving world. Conventional texts cannot be used by them and specialised things like books written in Braille script are much limited. An alternative, the text readers, available in the market are very expensive. Here lies the idea of project: implementing the idea of optical character recognition and integrating it with text to speech engine, a reader can be made. The accessibility is improvised by applying hard keys. The program is done on the small low-cost computer – Raspberry Pi, utilising the features of internet of things. Therefore, the aim of the project is to make a low cost, handy text book reader oriented for visually impaired students. The project is integrated with object labelling also.

INDEX

	Page No.
1. THE EVOLUTION OF ELECTRONICS	5
1.1. The Integration Eras	7
1.2. Fabrication of ICs	9
2. INTRODUCTION TO MICROPROCESSORS	13
2.1. Some Common Terms in Microprocessors	14
2.2. Brief History of Microprocessors	19
2.3. Microprocessor of Raspberry Pi	20
3. Internet of Things	22
3.1. Applications of IoT	23
3.2. Scope of IoT in this Project	24
4. HARDWARE SETUP	28
4.1. Raspberry Pi	28
4.2. Input Switches	33
4.3. Camera	34
4.4. Speaker	37
4.5. Power Supply	38
5. OPERATING SYSTEM AND OTHER DEPENDENCIES SET UP	40
5.1. Installation of Operating System	41
5.2. Programming the Raspberry Pi	43
5.3. Installation of Packages and Modules	44
5.3.1. Installing OpenCV	44
5.3.2. The OCR – Space API Package	48
5.3.3. Installation of Other Packages	50
6. CORE PROGRAMS AND THE ACTION OF Pi-Eye	52
6.1. The OCR Program: “OCRO.py”	53
6.2. The Object Detection Program: “Micro.py”	64
6.3. Controlling Programs	70
6.4. Action of the Device	74
CONCLUSION	76
Further Developments	76
REFERNCES	77

CHAPTER 1

THE EVOLUTION OF ELECTRONICS

In this 21st century, every day we are dealing with the electronic circuits and devices. For example, home appliances, computers, transport systems, cell phones, cameras, television, modern medical equipment, etc. all have electronic components and circuits. Today's world of electronics has made deep inroads in several areas, such as healthcare, medical diagnosis, automobiles, industries, food processing, agriculture and almost all fields of everyday life. Therefore, looking backwards to know the past and knowing the brief history of electronics is necessary to revive our minds and to get inspired by the eminent scholars.

The history of electronics is a story of the twentieth century and three key components: the vacuum tube, the transistor, and the integrated circuit. In 1883, Thomas Alva Edison discovered that electrons will flow from one metal conductor to another through a vacuum. This discovery of conduction became known as the Edison effect. In 1904, John Fleming applied the Edison effect in inventing a two-element electron tube called a diode, and Lee De Forest followed in 1906 with the three-element tube, the triode. These vacuum tubes were the devices that made manipulation of electrical energy possible so it could be amplified and transmitted. Subsequently, the transistor era began with the junction transistor invention in 1948. The three researchers of BELL laboratory, John Bardeen, Walter Brattain, and William Shockley invented the first bipolar transistor. The term “transistor” is coined by John R. Pierce. This is an abbreviated combination of the words, “transconductance" or "transfer", and "varistor". The varistor is a passive two-terminal solid state semiconductor device that is used to provide protection to electrical and electronic circuits. The device logically belongs in the varistor family, and has the transconductance or transfer impedance of a device having gain, so that this combination is descriptive. For the invention, they

received Nobel Prize in Physics in 1956.The use of germanium and silicon semiconductor materials made theses transistor gain the popularity and wide-acceptance usage in different electronics. Materials that contain both the properties of a conductor and an electrical insulator are known as semiconductors. Semiconductor devices were a suitable and efficient replacement for the thermionic devices (vacuum tubes). The invention of “transistor” is the beginning of “Modern electronics". Developments in "electronics" after the invention of "transistor” introduced an independent branch of engineering called "Electronics Engineering".

The subsequent years witnessed the invention of the integrated circuits (ICs) that drastically changed the electronic circuits' nature as the entire electronic circuit got integrated on a single chip, which resulted in low cost, small size and light-weight electronic devices. Jack Kilby of Texas Instruments made the first IC in 1958. During this time (in October 4, 1957), Soviet Union (USSR) launched the first artificial satellite to the space. The name of the satellite was ‘Sputnik’. Therefore, a large demand of highly efficient ICs was erupted. The years 1958 to 1975 marked the introduction of IC with enlarged capabilities of over several thousand components on a single chip. The trend further carried forward with the JFETs and MOSFETs that were developed during 1951 to 1958 by improving the device designing process and by making more reliable and powerful transistors. Digital integrated circuits were yet another robust IC development that changed the overall architecture of computers. All these radical changes in all these components led to the introduction of microprocessor in 1969 by Intel. Soon after, the analogue integrated circuits developed that introduced an operational amplifier for an analogue signal processing. These analogue circuits include analogue multipliers, ADC and DAC converters and analogue filters.

1.1 The Integration Eras

The first integrated circuits contained only a few transistors. Called "Small-Scale Integration" (SSI), they used circuits containing transistors numbering in the tens. SSI circuits were crucial to early aerospace projects, and vice-versa. Both the Minuteman missile and Apollo program needed lightweight digital computers for their inertial guidance systems; the Apollo guidance computer led and motivated the integrated-circuit technology, while the Minuteman missile forced it into mass-production. These programs purchased almost all of the available integrated circuits from 1960 through 1963, and almost alone provided the demand that funded the production improvements to get the production costs from $1000/circuit (in 1960 dollars) to merely $25/circuit (in 1963 dollars).[citation needed] They began to appear in consumer products at the turn of the decade, a typical application being FM inter-carrier sound processing in television receivers.

The next step in the development of integrated circuits, taken in the late 1960s, introduced devices which contained hundreds of transistors on each chip, called "Medium-Scale Integration" (MSI). They were attractive economically because while they cost little more to produce than SSI devices, they allowed more complex systems to be produced using smaller circuit boards, less assembly work (because of fewer separate components), and a number of other advantages.

Further development, driven by the same economic factors, led to "Large-Scale Integration" (LSI) in the mid-1970s, with tens of thousands of transistors per chip. Integrated circuits such as 1K-bit RAMs, calculator chips, and the first microprocessors, that began to be manufactured in moderate quantities in the early 1970s, had under 4000 transistors. True LSI circuits, approaching 10000 transistors, began to be produced around 1974, for computer main memories and second-generation microprocessors.

The next step in the development process, starting in the 1980s and continuing through the present, was "Very Large-Scale Integration" (VLSI). This could be said to start with hundreds of thousands of transistors in the early 1980s, and continues beyond several billion transistors. There was no single breakthrough that allowed this increase in complexity, though many factors helped. Manufacturing moved to smaller rules and cleaner fabrications, allowing them to produce chips with more transistors with adequate yield, as summarized by the International Technology Roadmap for Semiconductors (ITRS). Design tools improved enough to make it practical to finish these designs in a reasonable time. The more energy efficient CMOS replaced NMOS and PMOS, avoiding a prohibitive increase in power consumption. Better texts such as the landmark textbook by Mead and Conway helped schools educate more designers. In 1986 the first one-megabit RAM chips were introduced, which contained more than one million transistors. Microprocessor chips passed the million-transistor mark in 1989 and the billion-transistor mark in 2005.

To reflect further growth of the complexity, the term ULSI that stands for "Ultra-Large-Scale Integration" was proposed for chips of complexity of more than 1 million transistors.

Wafer-scale integration (WSI) is a system of building very-large integrated circuits that uses an entire silicon wafer to produce a single "super-chip". Through a combination of large size and reduced packaging, WSI could lead to dramatically reduced costs for some systems, notably massively parallel supercomputers. The name is taken from the term Very-Large-Scale Integration, the current state of the art when WSI was being developed.

System-on-a-Chip (SoC) is an integrated circuit in which all the components needed for a computer or other system, are included on a single chip. The design of such a device can be complex and costly, and building disparate components on a single piece of silicon may compromise the efficiency of some elements. However, these drawbacks are offset by lower

manufacturing and assembly costs and by a greatly reduced power budget: because signals among the components are kept on-die, much less power is required.

Three-Dimensional Integrated Circuit (3D-IC) has two or more layers of active electronic components that are integrated both vertically and horizontally into a single circuit. Communication between layers makes the power consumption, much lower than in equivalent separate circuits. Judicious use of short vertical wires can substantially reduce overall wire length for faster operation.

Moore's law refers to an observation made by Intel co-founder Gordon Moore in 1965. He noticed that the number of transistors per square inch on integrated circuits had doubled every year since their invention.

Moore's law predicts that this trend will continue into the foreseeable future. Although the pace has slowed, the number of transistors per square inch has since doubled approximately every 18 months. This is used as the current definition of Moore's law.

Because Moore's law suggests exponential growth, it is unlikely to continue indefinitely. Most experts expect Moore's law to hold for another two decades. Some studies show that the physical limitations will be achieved in this decade itself. Development of nanotechnology is expected to help the technological world to survive the saturation of Moore’s-law.

1.2 Fabrication of ICs

The semiconductors of the periodic table of the chemical elements were identified as the most likely materials for a solid-state vacuum tube by researchers like William Shockley at Bell Laboratories starting in the 1930s. Starting with copper oxide, proceeding to germanium, then silicon, the materials were systematically studied in the 1940s and 1950s. Today, silicon monocrystals are the main substrate used for integrated circuits (ICs) although some III-V compounds of the periodic table such as gallium arsenide are used for specialised

applications like LEDs, lasers, solar cells and the highest-speed integrated circuits. It took decades to perfect methods of creating crystals without defects in the crystalline structure of the semiconducting material. Semiconductor ICs are fabricated in a layer process which includes these key process steps:

i. Imaging.
ii. Deposition.
iii. Etching.

The main process steps are supplemented by doping, cleaning and planarisation steps. Monocrystal silicon wafers (or for special applications, silicon on sapphire or gallium arsenide wafers) are used as the substrate. Photolithography is used to mark different areas of the substrate to be doped or to have poly-silicon, insulators or metal (typically aluminium) tracks deposited on them.

- Integrated circuits are composed of many overlapping layers, each defined by photolithography, and normally shown in different colours. Some layers mark where various dopants are diffused into the substrate (called diffusion layers), some define where additional ions are implanted (implant layers), some define the conductors (polysilicon or metal layers), and some define the connections between the conducting layers (via or contact layers). All components are constructed from a specific combination of these layers.
- In a self-aligned CMOS process, a transistor is formed wherever the gate layer (polysilicon or metal) crosses a diffusion layer.
- Resistive structures, meandering stripes of varying lengths, form the loads on the circuit. The ratio of the length of the resistive structure to its width, combined with its sheet resistivity determines the resistance.
- Capacitive structures, in form very much like the parallel conducting plates of a traditional electrical capacitor, are formed according to the area of the

"plates", with insulating material between the plates. Owing to limitations in size, only very small capacitances can be created on an IC.

- More rarely, inductive structures can be built as tiny on-chip coils, or simulated by gyrators.

Since a CMOS (Complementary Metal-Oxide Semiconductor) device only draws current on the transition between logic states, CMOS devices consume much less current than bipolar devices.

A random-access memory is the most regular type of integrated circuit; the highest density devices are thus memories; but even a microprocessor will have memory on the chip. (See the regular array structure at the bottom of the first image.) Although the structures are intricate – with widths which have been shrinking for decades – the layers remain much thinner than the device widths. The layers of material are fabricated much like a photographic process, although light waves in the visible spectrum cannot be used to "expose" a layer of material, as they would be too large for the features. Thus, photons of higher frequencies (typically ultraviolet) are used to create the patterns for each layer. Because each feature is so small, electron microscopes are essential tools for a process engineer who might be debugging a fabrication process.

Each device is tested before packaging using automated test equipment (ATE), in a process known as wafer testing, or wafer probing. The wafer is then cut into rectangular blocks, each of which is called a die. Each good die (plural dice, dies, or die) is then connected into a package using aluminium (or gold) wires which are welded to pads, usually found around the edge of the die. After packaging, the devices go through final testing on the same or similar ATE used during wafer probing. Test cost can account for over 25% of the cost of fabrication on lower cost products, but can be negligible on low yielding, larger, and/or higher cost devices.

A fabrication facility (commonly known as a semiconductor fab) costs over a billion US Dollars to construct, because much of the operation is automated. The most advanced processes employ the following techniques:

- The wafers are up to 300 mm in diameter (wider than a common dinner plate).
- Use of 65 nanometer or smaller chip manufacturing process. Intel, IBM, NEC, and AMD are using 45 nanometers for their CPU chips, and AMD and NEC have started using a 65-nanometer process. IBM and AMD are in development of a 45 nm process using immersion lithography.
- Copper interconnects where copper wiring replaces aluminium for interconnects.
- Low-K dielectric insulators.
- Silicon on insulator (SOI)
- Strained silicon in a process used by IBM known as Strained silicon directly on insulator (SSDOI)

CHAPTER 2

INTRODUCTION TO MICROPROCESSORS

Digital systems are designed to store, process, and communicate information in digital form. They are found in a wide range of applications, including process control, communication systems, digital instruments, and consumer products. A digital computer, more commonly called simply a computer, is an example of a typical digital system.

A computer manipulates information in digital or more precisely, binary form. A binary number has only two discrete values: zero or one. Each discrete value is represented by the OFF and ON status of an electronic switch called a transistor. All computers understand only binary numbers. Any decimal number (base 10, with ten digits from 0 to 9) can be represented by a binary number (base 2, with digits 0 and 1).

The basic blocks of a computer are the central processing unit (CPU), the memory, and the input/output (I/O). The CPU of a computer is basically the same as the brain of a human being; so computer memory is conceptually similar to human memory. A question asked of a human being is analogous to entering a program into a computer using an input device such as a keyboard, and a person answering a question is similar in concept to outputting the program result to a computer output device such as a printer. The main difference is that human beings can think independently, whereas computers can only answer questions for which they are programmed. Computer hardware includes such components as memory, CPU, transistors, nuts, bolts, and so on. Programs can perform a specific task, such as addition, if the computer has an electronic circuit capable of adding two numbers. Programmers cannot change these electronic circuits but can perform tasks on them using instructions.

Computer software consists of a collection of programs that contain instructions and data for performing a specific task. All programs, written using any programming language

(e.g., C++, Python), must be translated into binary prior to execution by a computer because the computer understands only binary numbers. Therefore, a translator is necessary to convert such a program into binary and this is achieved using a translator program called a compiler. Programs in the binary form of 1's and 0's are then stored in the computer memory for execution. Also, as computers can only add, all operations, including subtraction, multiplications, and division are performed by addition.

Due to advances in semiconductor technology, it is possible to fabricate a CPU on a single chip. The result is a microprocessor. Both metal-oxide semiconductor (MOS) and bipolar technologies are used in the fabrication process. The CPU can be placed on a single chip when MOS technology is used. However, several chips are required with bipolar technology. At present, HCMOS (high-speed complementary MOS) or BICMOS (combination of bipolar and HCMOS) technology is normally used to fabricate a microprocessor on a single chip. Along with the microprocessor chip, appropriate memory and I/O chips can be used to design a microcomputer. The pins on each one of these chips can be connected to the proper lines on a system bus, which consists of address, data, and control lines. In the past, some manufacturers designed a complete microcomputer on a single chip with limited capabilities. Single-chip microcomputers were used in a wide range of industrial and home applications.

2.1 Some Common Terms in Microprocessors

Each and every microprocessor has mainly three parts, which are namely ALU, Control Unit (CU) and I/O. Some of the common terms and parameters involved in microprocessors are explained as follows:

- An Address is a pattern of 0's and 1's that represents a specific location in memory or a particular I/O device. Typical 8-bit microprocessors (eg: Intel 8085) have 16 address lines, and, these 16 lines can produce 216 unique 16-bit

patterns from 0000000000000000 to 1111111111111111, representing 65,536 different address combinations.

- Addressing mode is the manner in which the microprocessor determines the operand (data) and destination addresses during execution of an instruction.
- An Arithmetic-logic unit (ALU) is a digital circuit that performs arithmetic and logic operations on two n-bit digital words. The value of n can be 4, 8, 16, 32, or 64. Typical operations performed by an ALU are addition, subtraction, AND-ing, OR-ing, and comparison of two n-bit digital words. The size of the ALU defines the size of the microprocessor. For example, a 32-bit microprocessor contains a 32-bit ALU.
- Bit is an abbreviation for the term binary digit. A binary digit can have only two values, which are represented by the symbols 0 and 1, whereas a decimal digit can have 10 values, represented by the symbols 0 through 9. The bit values are easily implemented in electronic and magnetic media by two-state devices whose states portray either of the binary digits 0 and 1. Examples of such two-state devices are a transistor that is conducting or not conducting, a capacitor that is charged or discharged, and a magnetic material that is magnetized north to south or south to north.
- Bit size refers to the number of bits that can be processed simultaneously by the basic arithmetic circuits of a microprocessor. A number of bits taken as a group in this manner is called a word. For example, a 32-bit microprocessor can process a 32-bit word. An 8-bit word is referred to as a byte, and a 4-bit word is known as a nibble.
- A bus consists of a number of conductors (wires) organized to provide a means of communication among different elements in a microprocessor system. The conductors in a bus can be grouped in terms of their functions. A

microprocessor normally has an address bus, a data bus, and a control bus. Address bits are sent to memory or to an external device on the address bus. Instructions from memory, and data to/from memory or external devices, normally travel on the data bus. Control signals for the other buses and among system elements are transmitted on the control bus. Buses are sometimes bidirectional; that is, information can be transmitted in either direction on the bus, but normally in only one direction at a time.

- Cache Memory is a high-speed, directly accessible, relatively small, semiconductor read/write memory block used to store data/instructions that the microprocessor may need in the immediate future. It increases speed by reducing the number of external memory reads required by the microprocessor. Typical 32-bit microprocessors such as the Intel Pentium are provided with on-chip cache memory. Pentium II supports two levels of cache. These are L1 (Level 1 cache) and L2 (Level 2 cache) cache memories. The L1 cache (smaller in size) is contained inside the microprocessor while L2 cache (larger in size) is interfaced to the microprocessor. This two level cache enhances the performance of the microprocessor.
- A Complex Instruction Set Computer (CISC) contains a large instruction set. It is difficult to pipeline compared to RISC. Motorola 68020 is a CISC microprocessor.
- Clock is analogous to human heart beats. The microprocessor requires synchronization among its components, and this is provided by a clock or timing circuits.
- The instruction set of a microprocessor is a list of commands that the microprocessor is designed to execute. Typical instructions are ADD, SUBTRACT, and STORE. Individual instructions are coded as unique bit

patterns which are recognized and executed by the microprocessor. If a microprocessor has 3 bits allocated to the representation of instructions, the microprocessor will recognize a maximum of 23, or eight, different instructions. The microprocessor will then have a maximum of eight instructions in its instruction set. It is obvious that some instructions will be more suitable than others to a particular application. For example, if a microprocessor is to be used in a calculating mode, instructions such as ADD, SUBTRACT, MULTIPLY, and DIVIDE would be desirable. In a control application, instructions inputting digitized signals to the processor and outputting digital control variables to external circuits are essential. The number of instructions necessary in an application will directly influence the amount of hardware in the chip set and the number and organization of the interconnecting bus lines.

- Memory Management Unit (MMU) allows programmers to write programs much larger than could fit in the main memory space available to the microprocessor. The programs are simply stored in a secondary device such as a hard disk and portions of the programs are swapped into the main memory as needed for execution by the microprocessor. The MMU is implemented as on-chip hardware in typical microprocessors such as the Pentium.
- A microprocessor is the CPU of a microcomputer contained on a single chip, and must be interfaced with peripheral support chips in order to function. In general, a CPU contains several registers (memory elements), an ALU, and a control unit. Note that the control unit translates instructions and performs the desired task. The number of peripheral devices depends on the particular application involved and may even vary within an application. As the microprocessor industry matures, more of these functions are being integrated

onto chips, to reduce the system package count. In general, a microcomputer typically consists of a microprocessor (CPU) chip, input and output chips and memory chips in which programs (instructions and data) are stored. Note that a microcontroller, on the other hand, is implemented on a single chip containing typically a CPU, memory, I/O, a timer, and A/D and D/A converter circuits.

- Pipelining is a technique that overlaps instruction fetch (instruction read) with execution. This allows a microprocessor's processing operation to be broken down into several steps (dictated by the number of pipeline levels or stages) so that the individual step outputs can be handled by the microprocessor in parallel. Pipelining is often used to fetch the microprocessor's next instruction while executing the current instruction, which speeds up the overall operation of the microprocessor considerably.
- Random-access memory (RAM) is storage medium for groups of bits or words whose contents cannot only be read but can also be altered at specific addresses. A RAM normally provides volatile storage, which means that its contents are lost in case power is turned off. RAMs are fabricated on chips and have typical densities of 4096 bits to 1 megabit per chip. These bits can be organized in many ways: for example, as 4096-by-1-bit words or as 2048-by-8-bit words. RAMs are normally used for the storage of temporary data and intermediate results as well as programs that can be reloaded from a backup non-volatile source. RAMs are capable of providing large storage capacity, in the megabit range.
- Read-only memory (ROM) is storage medium for the groups of bits called words, and its contents cannot normally be altered once programmed. A typical ROM is fabricated on a chip and can store, for example, 2048 eight-bit

words, which can be accessed individually by presenting to it one of 2048 addresses. This ROM is referred to as a 2K by 8-bit ROM. 10110111 is an example of an 8-bit word that might be stored in one location in this memory. A ROM is a non-volatile storage device, which means that its contents are retained in case power is turned off. Because of this characteristic, ROMs are used to store programs (instructions and data) that must always be available to the microprocessor.

- A register can be considered as volatile storage for a number of bits. These bits may be entered into the register simultaneously (in parallel) or sequentially (serially) from right to left or from left to right, 1 bit at a time. An 8-bit register storing the bits 11110000 is represented as follows:

1	1	1	1	0	0	0	0

- A reduced instruction set computer (RISC) contains a simple instruction set. The RISC architecture maximizes speed by reducing clock cycles per instruction and makes it easier to implement pipelining. A Power PC is a RISC microprocessor.
- A Superscalar microprocessor is provided with more than one pipeline and can execute more than one instruction per clock cycle.

2.2 Brief History of Microprocessors

The Intel Corporation is generally acknowledged as the company that introduced the first microprocessor successfully into the marketplace. Its first processor, the 4004, was introduced in 1971 and evolved from a development effort while making a calculator chip set. The 4004 microprocessor was the central component in the chip set, which was called the MCS-4. The other components in the set were a 4001 ROM, a 4002 RAM, and a 4003-shift register.

The microprocessors introduced between 1971 and 1972 were the first-generation systems designed using PMOS (P-type metal-oxide-semiconductor) technology. In 1973, second-generation microprocessors such as the Motorola 6800 and the Intel 8080 (8-bit microprocessors) were introduced. The second-generation microprocessors were designed using NMOS (negative channel metal oxide semiconductor) technology. This technology resulted in a significant increase in instruction execution speed over PMOS and higher chip densities. Since then, microprocessors have been fabricated using a variety of technologies and designs. NMOS microprocessors such as the Intel 8085, the Zilog Z80, and the Motorola 6800/6809 were introduced based on second-generation microprocessors. A third generation HMOS (high-speed MOS) microprocessor, introduced in 1978 is typically represented by the Intel 8086 and the Motorola 68000, which are 16-bit microprocessors.

During the 1980's, fourth-generation HCMOS (High Density Complimentary MOS) and BICMOS (a combination of bipolar and HCMOS) 32-bit microprocessors evolved. Intel introduced the first commercial 32-bit microprocessor, the problematic Intel 432, which was eventually discontinued. Since 1985, more 32-bit microprocessors have been introduced. These include Motorola's 68020, 68030, 68040, 68060, PowerPC, Intel's 80386, 80486, the Intel Pentium family, Core Duo, and Core2 Duo microprocessors. The 64-bit microprocessors such as Power PC 750 include all the features of 32-bit microprocessors. In addition, they contain multiple on-chip integer and floating-point units and a larger address and data buses. The 64-bit microprocessors can typically execute four instructions per clock cycle and can run at a clock speed of over 2 GHz.

2.3 Microprocessor of Raspberry Pi

In this project the main hardware is the small computer, namely “Raspberry Pi” which has a 64-bit processor. The CPU specification is given as follows: 1.2GHz 64-bit quad-core ARMv8 Cortex-A53. An ARM processor is one of a family of CPUs based on

the RISC (reduced instruction set computer) architecture developed by Advanced RISC Machines (ARM). The Cortex-A53 processor delivers significantly more performance than its predecessors at a higher level of power efficiency. This takes the performance of the core above that of the Cortex-A7 processor, which defines many popular mainstream and entry-level mobile platforms. The performance graph to the right shows the performance improvements of the Cortex-A53 processor against the Cortex-A7 processor. The ARMv8-A architecture brings a number of new features. These include 64-bit data processing, extended virtual addressing and a 64-bit general purpose registers. The Cortex-A53 processor is Arm's first Armv8-A processor aimed at providing power-efficient 64-bit processing. The Cortex-A53 is a superscalar processor, capable of dual-issuing some instructions. A scalar processor that can execute at most one single instruction per clock cycle, a superscalar processor can execute more than one instruction during a clock cycle by simultaneously dispatching multiple instructions to different execution units on the processor.

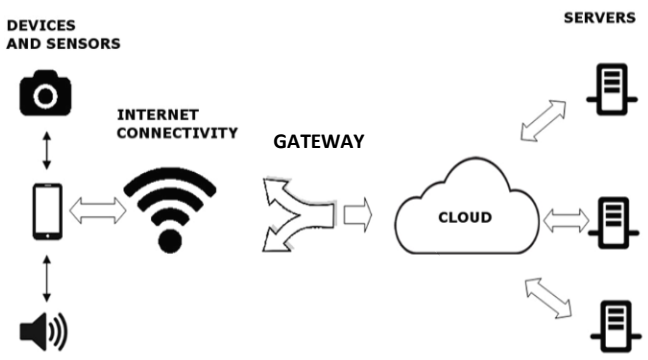
The 900MHz 32-bit quad-core ARM Cortex-A7 CPU complex in Raspberry Pi 2 has been replaced by 1.2GHz 64-bit quad-core ARM Cortex-A53 on February 2016. Combining a 33% increase in clock speed with various architectural enhancements, this provides a 50-60% increase in performance. Thus, Raspberry pi 3, involved in this project is essentially a 64-bit processor.

CHAPTER 3

INTERNET OF THINGS

The world is now finding its pace through the modern generation of technology, which is named as Internet of Things (IoT). Broadband Internet is become more widely available, the cost of connecting is decreasing, more devices are being created with Wi-Fi capabilities and sensors built into them, technology costs are going down, and modern devices' penetration is sky-rocketing. All of these things are creating a "perfect storm" for the IoT. The IoT is a giant network of connected "things". Things can be persons as well as devices.

IoT consists of all the web-enabled devices that collect, send and act on data they acquire from their surrounding environments using embedded sensors, processors and communication hardware. These devices, often called "connected" or "smart" devices, can sometimes talk to other related devices, a process called machine-to-machine (M2M) communication, and act on the information they get from one another. Humans can interact with the gadgets to set them up, give them instructions or access the data, but the devices do most of the work on their own without human intervention. A block diagram showing a basic IoT process is given as figure 1:



IoT; Basic Action Diagram (Fig 3.1)

According to Jacob Morgan of Forbes magazine, the new rule for the future is going to be, "Anything that can be connected, will be connected." It seems to be similar to the motto of globalization: “Think globally, act locally”. Actually, the device senses or reads needed data according to its design, manipulates the input, send the data to the designated cloud platform which analyzes and transfer the data towards next designated device or send back the result. The above processes can be said as the cross section of action of the internet of things.

Connected devices also generate massive amounts of Internet traffic, including loads of data that can be used to make the devices useful, but can also be mined for other purposes. All this new data, and the Internet-accessible nature of the devices, raises both privacy and security concerns. But, the versatility of fabulous applications of this next generation technology IoT is very much acceptable.

3.1 Applications of IoT

Let us go through some applications of internet of things in order to know the potential of it. For example, consider an intelligent refrigerator which can detect the reduced stock of vegetables in it and order for the needed vegetables online. We can imagine same feature having container, say, which detects the sugar or rice levels in it and order for the same online. If some online cash payment method is linked with the network, then the user is interfered in none of the above processes. Let us go through another example which is already familiar to so many. A wearable device which can detect one heart rate, blood pressure, sleep deepness and other health parameters, analyses the data and make suggestions to apply in user’s food habits, exercise and overall life. It can even diagnose and predict an upcoming heart attack or stroke and call for emergency medical help itself, without a prompt by the user. The internet of thing is thus related to Artificial Intelligence (AI) also. Another example is, in some super markets in USA, there is no person to make bills for one’s

shopping items. Detectors placed at the exit doors of supermarkets detect the items on one's cart, make bill and deduct the sum of prices from the user's linked electronic-wallet or bank account. Smart appliances like smart lighting and other home appliances which can be controlled at a distance are also coming forward. Thus, IoT makes life simpler and complex things, easier and faster.

IoT finds applications in large scale also. Automated industries with machines, fault detectors and error correctors utilizing IoT is one of it. Enabling IoT in industries can improve processes to increase productivity and reduce material waste and unforeseen downtime. IoT find its role in power management. Efficient energy management features also can be derived using IoT. An example is, applying IoT to make power grids smarter. Localized small scale power grids can be applied to implement house hold solar power stations; therefore, we can exempt battery which is the main factor of expensiveness solar electricity systems. In oil and gas excavation industry, applying IoT can improve operational visibility. Automobile and public transportation fields also find versatile applications of IoT. For example, self-driving vehicles which are exalted as the super kid of artificial intelligence also use IoT in detecting the current road, traffic, weather and other conditions. Common modern vehicles also use IoT in automated vehicle management systems. Sensors in city infrastructure can help reduce road congestion and warn us when infrastructure is in danger of crumbling. Gadgets out in the open can monitor for changing environmental conditions and warn us of impending disasters. More or less IoT makes the globe, a hassle-free smart-super globe.

3.2 Scope of IoT in this Project

This project is actually based on the features of the technology of next era, the internet of things. IoT is often referred as Internet of Everything (IoE) also. In this project the person one who uses the device do not directly connect himself to internet. But the device, "The Pi-

Eye" is connected to the internet and does the Optical Character Recognition (OCR) as well as labelling of object. The text to speech engine (TTS) is also utilising the feature of IoT. OCR is done by a cloud platform API, namely "OCR-Space". Labelling job is done by the technology giant Microsoft's API, with its platform name as "Azure" and the TTS work is done by internet beast "Google".

API is the abbreviation of application program (or programming) interface. API is arguably what really ties together the connected "things" of the "internet of things." IoT APIs are the points of interaction between an IoT device and the internet and/or other elements within the network. The device connects to the Internet and sends data to a server. The server then retrieves that data, interprets it, performs the necessary actions and sends it back to your phone. The application then interprets that data and presents you with the information you wanted in a readable way. This is done in a process involving an API. The API technology makes IoT meaningful as well as more secure, as only the needed small packets of data is shared across devices or between cloud server and the device.

The APIs used in project are accessed by corresponding API subscription keys, which are provided by the vendors.

The free OCR-Space API provides a simple way of parsing images and getting the extracted text results returned in a JSON format. JSON is the abbreviation of Java-Script Object Notation. When exchanging data between a browser and a server, the data can only be text. JSON is text, and we can convert any JavaScript object into JSON, and send JSON to the server. We can also convert any JSON received from the server into JavaScript objects. This way we can work with the data as JavaScript objects, with no complicated parsing and translations. Here the OCRed results are converted from JSON format to text format, using the python module, namely "simple json". The conversion also involved some sort of logical thinking which is up to the project scholars. The free version of OCR-Space API is used in this project, which allows user to do OCR on a maximum number of 25,000

images per month at a maximum rate of 500 images per day. The maximum limits are seeming to be more than adequate than the scope of the project, “the Pi-Eye”.

The object labelling program of the project uses Microsoft’s Computer Vision API. This feature returns information about visual content found in an image. Use tagging, descriptions and domain-specific models to identify content and label it with confidence. Apply the adult/racy settings to enable automated restriction of adult content. Identify image types and colour schemes in pictures. You can use the Analyze Image method to extract visual features based on image content. Either uploads an image or specifies an image URL, and then selects which features to return:

- A detailed list of tags related to the image content.
- A description of image content as a complete sentence.
- The coordinates, gender, and age of any faces that are in the image.
- The Image Type value (clip art or a line drawing).
- The dominant colour of the image, the accent colour, or whether an image is black and white.
- The category taxonomy.
- Whether the image contains adult or sexually suggestive content.

In this project the device is getting response from the API in JSON format, including the above listed analyzed parameters. The program just extracts the description of image content as a complete sentence only, which is then read out. The above API is a paid one. The project is actually built for the text book reading purpose and the detection and labelling of objects is only a further development of the project. Therefore, the trial API version of Microsoft is used here. This allows one month subscription with a maximum limit of analysis of 5,000 pictures per month.

The third application of IoT in the project is earlier seen text to speech feature provided by Google. The text to speech engine is also referred as ‘speech synthesizer’. The

TTS engine from Google is abbreviated as GTTS, which is a free API. This engine helps us to create an mp3 file from spoken text via the Google TTS (Text-to-Speech) API, where the text is uploaded to the cloud. Google's TTS engine has got a python wrapper or module, namely "gtts", so that we can just import the module to our program to connect with the API. The GTTS is available in the following languages:

	Code	Language
1	'af'	'Afrikaans'
2	'sq'	'Albanian'
3	'ar'	'Arabic'
4	'hy'	'Armenian'
5	'bn'	'Bengali'
6	'ca'	'Catalan'
7	'zh'	'Chinese'
8	'zh-cn'	'Chinese (Mandarin/China)'
9	'zh-tw'	'Chinese (Mandarin/Taiwan)'
10	'zh-yue'	'Chinese (Cantonese)'
11	'hr'	'Croatian'
12	'cs'	'Czech'
13	'da'	'Danish'
14	'nl'	'Dutch'
15	'en'	'English'
16	'en-au'	'English (Australia)'
17	'en-uk'	'English (United Kingdom)'
18	'en-us'	'English (United States)'
19	'eo'	'Esperanto'
20	'fi'	'Finnish'
21	'fr'	'French'
22	'de'	'German'
23	'el'	'Greek'
24	'hi'	'Hindi'
25	'hu'	'Hungarian'
26	'is'	'Icelandic'
27	'id'	'Indonesian'
28	'it'	'Italian'
29	'ja'	'Japanese'
30	'km'	'Khmer (Cambodian)'
31	'ko'	'Korean'
32	'la'	'Latin'
33	'lv'	'Latvian'
34	'mk'	'Macedonian'
35	'no'	'Norwegian'
36	'pl'	'Polish'
37	'pt'	'Portuguese'
38	'ro'	'Romanian'
39	'ru'	'Russian'
40	'sr'	'Serbian'
41	'si'	'Sinhala'
42	'sk'	'Slovak'
43	'es'	'Spanish'
44	'es-es'	'Spanish (Spain)'
45	'es-us'	'Spanish (United States)'
46	'sw'	'Swahili'
47	'sv'	'Swedish'
48	'ta'	'Tamil'
49	'th'	'Thai'
50	'tr'	'Turkish'
51	'uk'	'Ukrainian'
52	'vi'	'Vietnamese'
53	'cy'	'Welsh'

We can also adjust the speech output speed by adding a request command to the API client program.

CHAPTER 4

HARDWARE SETUP

The project mainly consists of five hardware parts:

1. Raspberry Pi – the computer chip
2. Input Switches
3. Camera
4. Speaker or Headphone
5. Power Supply (power bank)

Each part is separately discussed along this chapter.

4.1 Raspberry Pi

Raspberry Pi is the main part of the project, which is essentially a computer chip. The computer is often called as “credit card sized computer” as its dimensions are (8.9cmx5.6cm), which almost similar to the actual credit card size (8.56cm×5.39cm). Creator Eben Upton's goal was to create a low-cost device that would improve programming skills and hardware understanding at the pre-university level. But thanks to its small size and accessible price, it was quickly adopted by tinkerers, makers, and electronics enthusiasts for projects that require more than a basic microcontroller (such as Arduino devices). It was developed by the Raspberry Pi Foundation, which is a UK registered charity (Registration Number 1129409). The foundation exists to promote the study of computer science and related topics, especially at school level, and to put the fun back into learning computing. The device is expected to have many other applications both in the developed and the developing world. Thus, this easily programmable computer chip finds role of protagonist in this project.

Birth of Raspberry pi happened in 2011. It is a product of eruption of revolutionary developments in semiconductor technology and mobile device technology. The first

generation (Raspberry Pi 1 Model B) was released in February 2012, followed by the simpler and cheaper Model A. In 2014, the Foundation released a board with an improved design, Raspberry Pi 1 Model B+. These boards are approximately credit-card sized and represent the standard mainline form-factor. Improved A+ and B+ models were released a year later. A "Compute Module" was released in April 2014 for embedded applications. The Raspberry Pi 2 which added more RAM was released in February 2015.

A Raspberry Pi Zero with smaller size and reduced input/output (I/O) and general-purpose input/output (GPIO) capabilities was released in November 2015 for US$5. By 2017, it became the newest mainline Raspberry Pi. On 28 February 2017, the Raspberry Pi Zero W was launched, a version of the Zero with Wi-Fi and Bluetooth capabilities, for US$10. On 12 January 2018, the Raspberry Pi Zero WH was launched, the same version of the Zero W with pre-soldered GPIO headers.

Raspberry Pi 3 Model B was released in February 2016 with a 64 bit quad core processor, and has on-board Wi-Fi, Bluetooth and USB boot capabilities. The latest model 3B+ appeared with a faster 1.4 GHz processor and a 3 times faster network based on gigabit Ethernet (300 Mbit / s) or 2.4 / 5 GHz dual-band Wi-Fi (100 Mbit / s). Other options are: Power over Ethernet (PoE), USB boot and network boot (an SD card is no longer required). This allows the use of the Pi in hard-to-reach places (possibly without electricity). In this project, the latest Raspberry Pi 3 Model B is used, which costs about ₹2900.

Raspberry Pi chip is essentially a Linux based computer. It is an embedded device consisting of microprocessor, RAM, GPIO (General Purpose Input/output pins), USB ports etc. The key specifications of Raspberry Pi 3 Model B are given as follows:

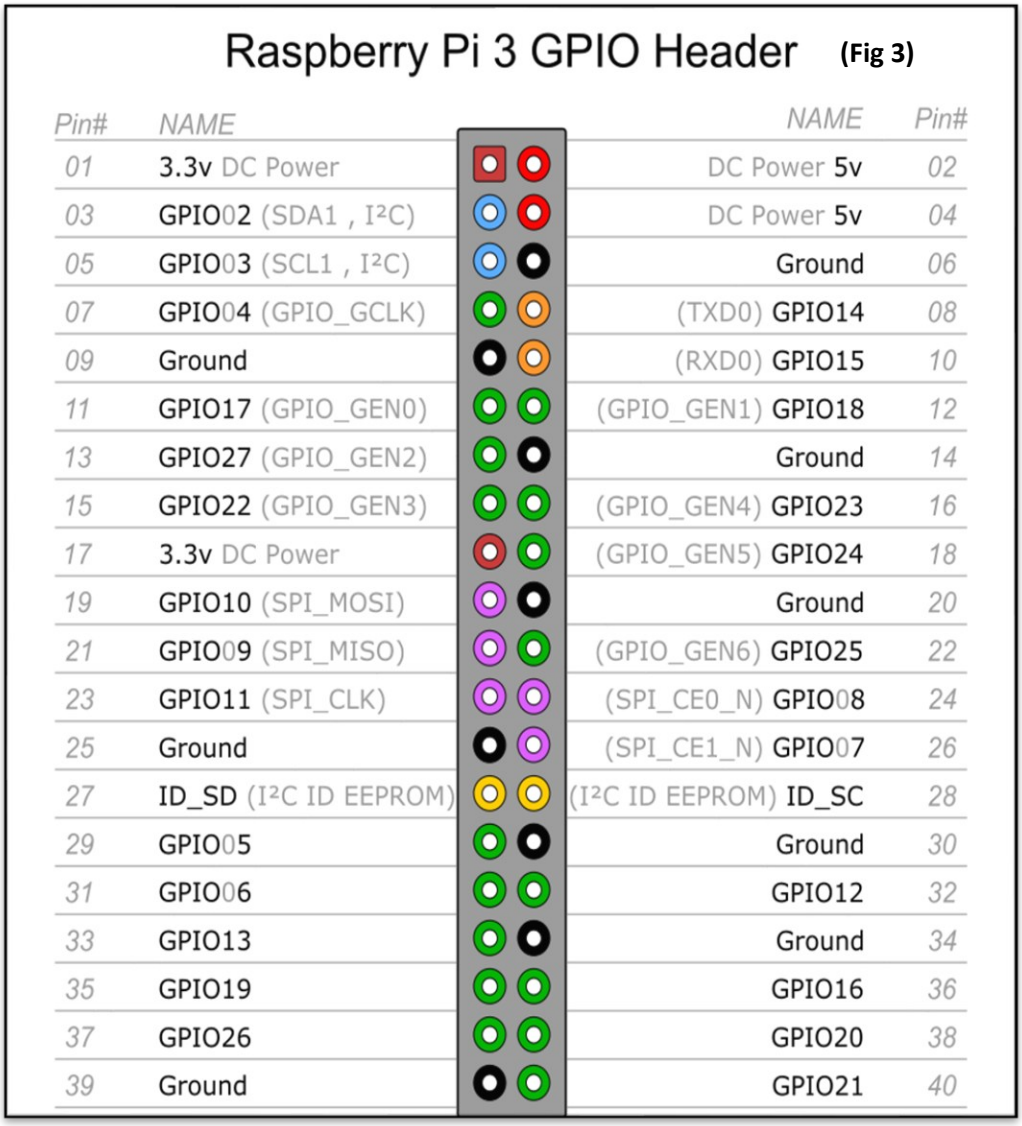
- Quad Core 1.2GHz Broadcom BCM2837 64bit CPU
- 1GB RAM

- BCM43438 wireless LAN and Bluetooth Low Energy (BLE – Bluetooth 4.1) on board
- 10/100 wired Ethernet RJ45
- 40-pin extended GPIO
- 4 USB 2.0 ports (via integrated USB hub in LAN9514)
- 4 Pole stereo output (3.5 mm jack audio output) and composite video port
- Full size HDMI (High-Definition Multimedia Interface)
- CSI camera port for connecting a Raspberry Pi camera
- DSI display port for connecting a Raspberry Pi touch-screen display
- Micro SD port for loading your operating system and storing data
- Micro USB power source up to 2.5A
- GPU - Broadcom VideoCore IV, OpenGL ES 2.0, OpenVG 1080p60 H.264 high-profile encode/decode, 400 MHz
- Power Ratings - 800mA (4.0 W), up to 2.5A.
- Size - 85.6 x 56.5 mm x 17mm
- Weight - 45g

The Raspberry Pi 3, with a quad-core ARM Cortex-A53 processor, is described as 10 times the performance of a Raspberry Pi 1. This was suggested to be highly dependent upon task threading and instruction set use. Benchmarks showed the Raspberry Pi 3 to be approximately 80% faster than the Raspberry Pi 2 in parallelised tasks.

Raspberry Pi 3 Model B (Fig 2)

A powerful feature of the Raspberry Pi is the row of GPIO (general-purpose input/output) pins along the top edge of the board. A 40-pin GPIO header is found on all current Raspberry Pi boards, including Raspberry Pi Model B. Input pins act like a voltage meter. They measure the voltage that comes in. Software PWM (pulse-width modulation) is available on all pins and hardware PWM is available on GPIO12, GPIO13, GPIO18, GPIO19 Digital pins will send 0 for a voltage under half of the maximum expected and 1 for values above the half. Analogue pins, on the other hand, will send values from 0 to 1024, depending on what the respective sensor is detecting. The programmer can assign the task to be performed according to the state of each GPIO pin.



Raspberry Pi can be programmed using Python language which is explained on upcoming chapter. Raspberry Pi can be programmed directly by connecting peripherals like monitor using HDMI cable, keyboard and mouse using USB ports. The programming can also be done by connecting the pi directly to a desktop computer or a laptop computer through the Ethernet port. The computer should be installed with an SSH (Secure Socket Shell) software, for example PuTTY. PuTTY is an SSH and telnet client. SSH is a software package that enables secure system administration and file transfers over insecure networks. The SSH protocol uses encryption to secure the connection between a client and a server. All user authentication, commands, output, and file transfers are encrypted. Most of the programming of the Raspberry Pi in this project was done using SSH protocol. Here, SSH

was done using the Wireless Local Area Network (WLAN) facility, i.e. Wi-Fi, and connecting the board to programmer's android mobile-phone. The smart-phone used was installed with an SSH software named, 'Rasp-Controller', which is developed by Ettore Gallina. While using this mode of programming, the phone was acting as 'Wireless Fidelity (Wi-Fi) Hotspot.'

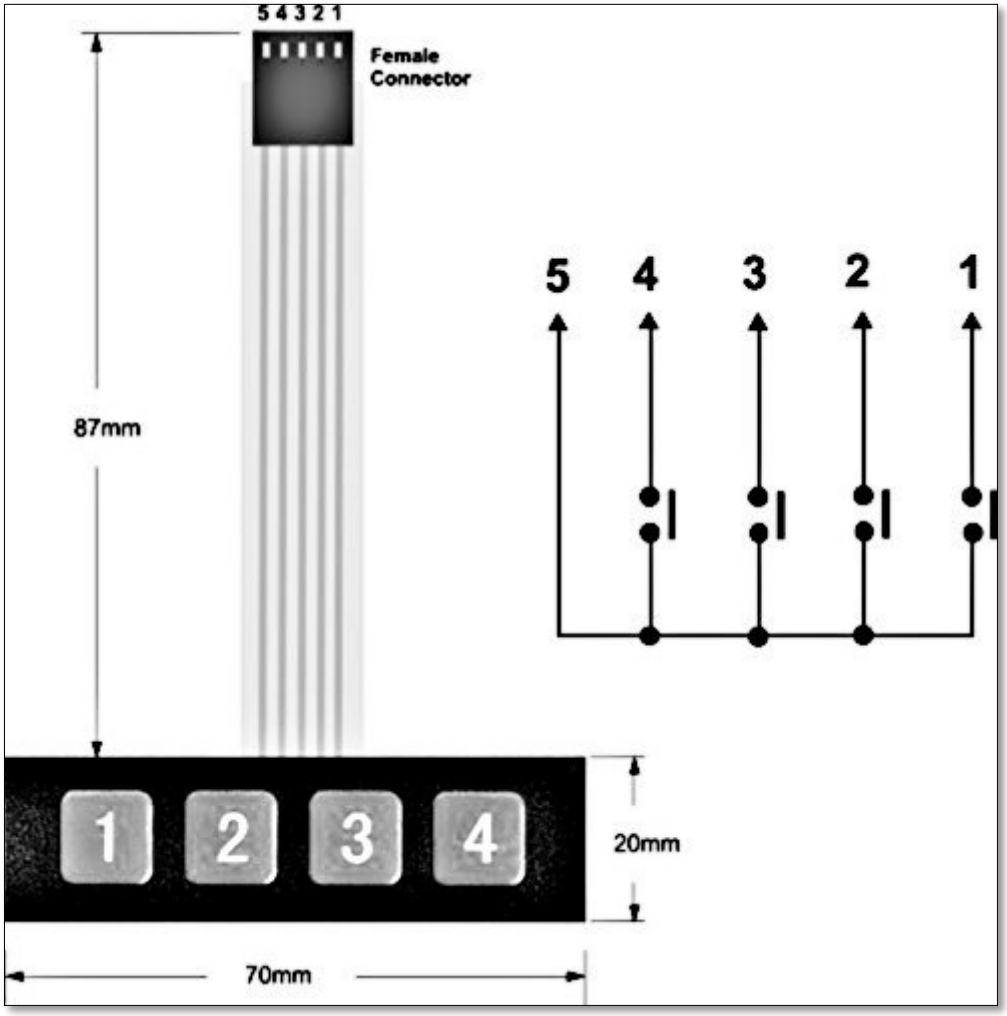
The embedded chip itself does not have a storage memory and therefore it comes without any operating system. The chip has got a Micro SD (Secure Digital) card slot. An 8 GB card is adequate for minimum scale operations with Raspberry Pi. In this project we are using a 16 GB, class 10 micro-SD card.

4.2 Input Switches

The project is designed for visually impaired persons. Therefore, hard keys should be added in the device to perform the specific tasks, assigned. Here, the device is connected with a 1x4 membrane keypad, which consists of 4 keys. It differs from a mechanical switch, which is usually made of copper and plastic parts: a membrane switch is a circuit printed on PET (Poly Ethelene Terephthalate or Poly(ethyl benzene-1,4-dicarboxylate)) or ITO (Indium Tin Oxide). The ink used for screen printing is usually copper / silver / graphite filled and therefore conductive. The keypad comes with a connection wires with a female jumper connecter, so we can connect it directly through the GPIO pins on the Raspberry Pi board. The specifications of membrane switch used in this project are given as follows:

- Pad Size: 104 x 24 x 0.8mm
- Cable Length: 96 mm (include connector)
- Weight: 6 gm.
- Max. Circuit Rating: 35 VDC, 100 mA.
- Insulation Spec: 100M Ohm, 100V

- Life Expectancy: 1 million closures
- Operation Temperature: -20 to +40 °C



1X4 Membrane Switch and Wiring Diagram (Fig 4)

The switch has 5 female connecter heads as shown in the figure. These are connected to the GPIO according to the design. The action of switch is explained in upcoming chapter.

4.3 Camera

Camera is an essential part of the project. The Raspberry Pi Camera Module is a custom designed add-on for Raspberry Pi. It attaches to Raspberry Pi by way of one of the small sockets on the board upper surface. This interface uses the dedicated CSI (Camera Serial Interface) interface, designed especially for interfacing to cameras.

Here we use a 5MP camera, namely Pi Camera version 1.3. The board itself is tiny, at around 25mm x 20mm x 9mm. It also weighs just over 3g, making it perfect for mobile or other applications where size and weight are important. It connects to Raspberry Pi by way of a short ribbon cable. The sensor itself has a native resolution of 5 megapixel, and has a fixed focus lens onboard. In terms of still images, the camera is capable of 2592 x 1944 pixel static images, and also supports 1080p30, 720p60 and 640x480p60/90 video. The camera cannot record audio, while taking a video. The camera employs a 5MP Omnivision 5647 camera sensor. The camera costs about ₹600.

The factory setting of the camera is to focus at infinity (∞). So, the object, whose image to be captured should be at least 5-meter distance away from the camera. But this is impractical in the case of our project, which is mainly designed to scan text book pages. The factory setting is somewhat fixed. So, we had to make the fixed focus, adjusted. It was done with a very great concentration, and successfully and precisely adjusted the focus by rotating the lens. Pictures were captured each time of rotation in a tiny angle to make information whether the focus has been adjusted correctly.

The default camera properties and other parameters are listed below:

- Sharpness of the image: Default value is 0. Can vary in the interval (-100,100).
- Contrast of the image: Default value is 0. Can vary in the interval (-100,100).
- Brightness of the image: Default value is 50. Can vary in the interval (0,100).
- Colour Saturation of the image: Default value is 0. Can vary in the interval (-100,100).
- Capture ISO (ISO stands for International Standards Organization, and it is a standardized industry scale for measuring sensitivity to light) can be varied from 100.
- Exposure value (EV) can be set between (-10,10).

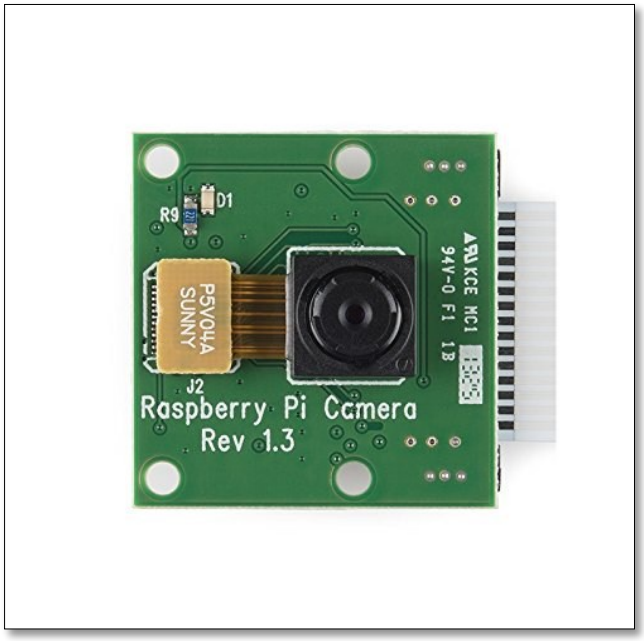
- There are several exposure modes as follows:
  - auto: use automatic exposure mode
  - night: select setting for night shooting
  - night preview: night mode 1
  - backlight: select setting for backlit subject
  - spotlight:
  - sports: select setting for sports (fast shutter etc.)
  - snow: select setting optimised for snowy scenery
  - beach: select setting optimised for beach
  - verylong: select setting for long exposures
  - fixedfps: constrain fps to a fixed value
  - antishake: anti-shake mode
  - fireworks: select setting optimised for fireworks
- White Balancing Mode can be toggle between Automatic mode and manual mode. The manual mode has several options as follows:
  - sun: sunny mode (between 5000K and 6500K)
  - cloud: cloudy mode (between 6500K and 12000K)
  - shade: shade mode
  - tungsten: tungsten lighting mode (between 2500K and 3500K)
  - fluorescent: fluorescent lighting mode (between 2500K and 4500K)
  - incandescent: incandescent lighting mode
  - flash: flash mode
  - horizon: horizon mode
- Metering modes available in the camera are:
  - average: average the whole frame for metering
  - spot: spot metering

- backlit: assume a backlit image
- matrix: matrix metering

❖ Rotation of image can be done in angles 90^0, 180^0, 270^0.

❖ Shutter speed can be specified in microseconds, with an upper limit of 6 seconds.

The above parameters can be designed according to the optimum working environmental conditions of the project.



Raspberry Pi Camera Module (Fig 5).

4.4 Speaker

As we seen earlier, the project is designed with concerning the visually impaired readers and main part of the project is to give speech output, speaker is an essential component. The project is single person oriented, so that an ordinary headphone is used to output the audio.

The computer chip used in this project, the Raspberry Pi 3 Model B comes along with a 3.5mm audio jack, which is the standard jack size for computers and mobile phones for outputting audio. So we can simply use any earphone or headphone with Raspberry Pi. If the

audio output is to be listened by a group of people, other common speakers can be used with this. But when it is to be done like that, the user should be aware of the fact that without an external power supply to the speaker, the Raspberry Pi cannot drive it and situations like this would destroy the board.

4.5 Power Supply

The board takes fixed 5 V DC input. This permits adoption of the micro-USB form factor, which, in turn, prevents the user from inadvertently plugging in out-of-range power inputs; that would be dangerous, since the 5 V would go straight to HDMI and output USB ports, even though the problem should be mitigated by some protections applied to the input power: The board provides a polarity protection diode, a voltage clamp, and a self-resetting semiconductor fuse. A power supply that can provide enough current to power the device plus any connected peripherals, and taking into account inefficiencies of the supply itself and the cable between the power supply and Raspberry Pi. Opting for a power supply that can supply at least 1 A if using USB peripherals or Pi plates that draw more than a few tens of mill amperes of current.

As the 5 V rail is brought out in the GPIO pins, user can power the Raspberry Pi from there too. You should mind however, that those are behind the power protection circuitry, so it is risky if we have not made a protection circuit.

The Raspberry Pi 3 is powered by a +5.1V micro-USB supply. Typically, the model B uses between 700-1000mA depending on what peripherals are connected; the model A can use as little as 500mA with no peripherals attached. The maximum power the Raspberry Pi can use is 1 Amp. If you need to connect a USB device that will take the power requirements above 1 Amp, then you must connect it to an externally-powered USB hub. Peripherals like

HDMI using monitor/TV or other displays more current is drained from the board. So, it is safe to use a power supply giving a ~2.0 A direct current.

In this project, a power bank is used as power supply hub. The power bank outputs USB 5V, 2.1A direct current. The battery has the capacity of 13,000mAh/3.7V, i.e. 48.1Wh.

Power Bank (Fig. 6)

CHAPTER 5

OPERATING SYSTEM AND OTHER DEPENDENCIES' SET UP

Raspberry Pi is essentially an embedded computer board. Therefore, the chip can be loaded with so many operating systems. Raspberry Pi can be made a custom-built gadget as in this project or a simple personal computer. The list of operating systems that can be booted on Raspberry Pi listed below:

- Raspbian
- Ubuntu MATE
- Windows 10 IoT
- Snappy Ubuntu
- Pidora
- Linutop
- SARPi
- Arch Linux ARM
- Gentoo Linux
- Free BSD
- Kali Linux
- RISC OS Pi
- Retro Pi
- Libreelec
- PiNET
- Open-Source Media Centre (OSMC)

The first one of the list - 'Raspbian' is the official operating system released. The project is also utilising the same OS. Raspbian is a free operating system based on Debian

optimized for the Raspberry Pi hardware. An operating system is the set of basic programs and utilities that make your Raspberry Pi run. However, Raspbian provides more than a pure OS: it comes with over 35,000 packages; pre-compiled software bundled in a nice format for easy installation on your Raspberry Pi. The initial build of over 35,000 Raspbian packages, optimized for best performance on the Raspberry Pi, was completed in June of 2012. However, Raspbian is still under active development with an emphasis on improving the stability and performance of as many Debian packages as possible.

5.1 Installation of Operating System

The operating system, Raspbian's image file is downloaded from the official site. The Raspbian with Raspberry Pi Desktop image contained in the ZIP archive is over 4GB in size and uses the ZIP64 format. To uncompress the archive, a unzip tool that supports ZIP64 is required. Now we need to use an image writing tool to install the image you have downloaded on your SD card. 'Etcher' is a graphical SD card writing tool that works on Mac OS, Linux and Windows, and is the easiest option for most users. Etcher also supports writing images directly from the zip file, without any unzipping required.

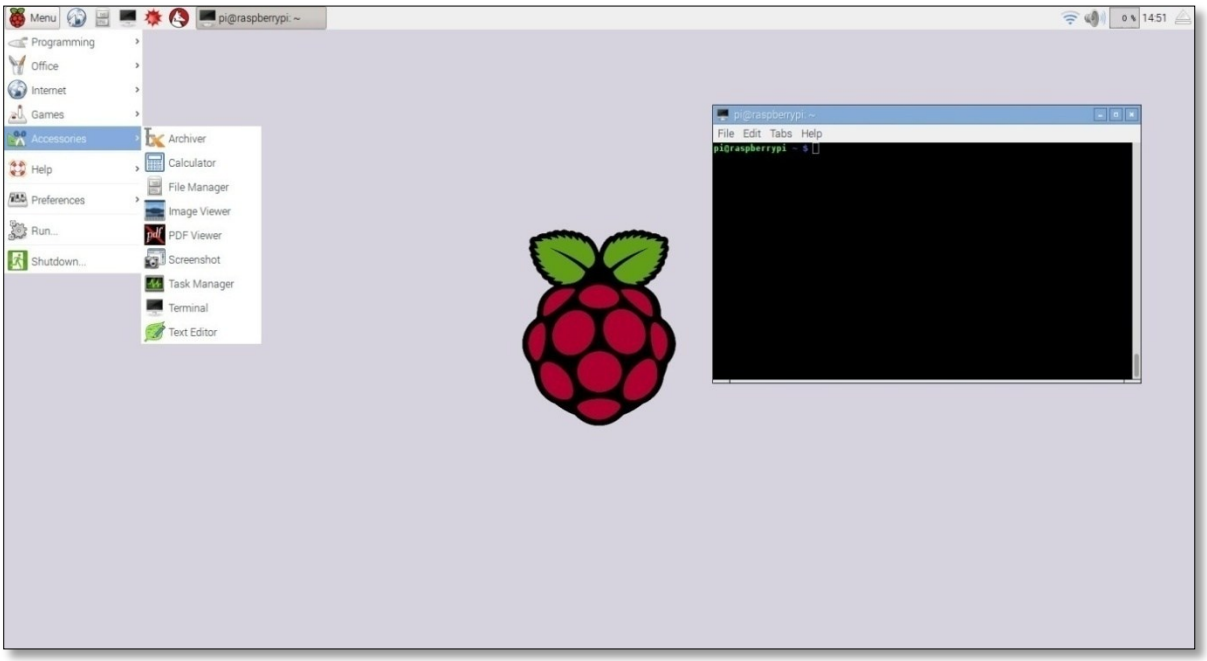
An alternative to 'Etcher' on windows platform is 'Win32 Disk Manager', which was used in the first steps of the project. The procedure to write Raspbian.img file on the SD card is given below:

- Insert the SD card into your SD card reader. You can use the SD card slot if you have one, or an SD adapter in a USB port. Note the drive letter assigned to the SD card. You can see the drive letter in the left-hand column of Windows Explorer, for example **G:**
- Download the Win32DiskImager utility from the Sourceforge Project page as an installer file, and run it to install the software.
- Run the Win32DiskImager utility from your desktop or menu.

- Select the image file you extracted earlier.
- In the device box, select the drive letter of the SD card. Be careful to select the correct drive: if you choose the wrong drive, you could destroy the data on your computer's hard disk! If you are using an SD card slot in your computer, and can't see the drive in the Win32DiskImager window, try using an external SD adapter.
- Click 'Write' and wait for the write to complete.
- Exit the imager and eject the SD card.

Now the bootable micro-SD card is ready. It is now inserted to the Raspberry Pi, which is connected with a monitor, keyboard and a mouse, which are essential. The installation process is now onwards very easy, as NOOBS, an installer helper is included.

As earlier seen, Raspbian consists of almost all needed package files for normal programming and computing processes. Specialised package files needed for the project is then installed on the device.



Desktop of Raspbian with Terminal (Fig 7)

5.2 Programming the Raspberry Pi

Raspbian comes preloaded with Python, the official programming language of the Raspberry Pi and IDLE 3, a Python Integrated Development Environment. The programmer can use the Python IDE to make programs to execute on Pi or use the 'Terminal' to run command by command. The terminal can also be used for making changes in the system configurations, such as making the camera port enabled, toggling of audio output source between HDMI port and the 3.5 mm audio jack or making the SSH active or inactive etc.

Programming can be directly done on Raspberry Pi using the monitor, keyboard and mouse connected to it or using the SSH (Secure Socket Shell) which is explained in the aforesaid chapter. Most part of the programming involved in this project is done using the SSH. The chip was connected to the SSH software in android mobile using Wi-Fi networking. SSH can be started by just running the below command or making changes in the configurations.

```
$ sudo service ssh start
```

As we have before seen, the programs are written in Python language. It was released in 1991. This object-oriented high-level programming language was designed by 'Guido van Rossum.' According to Tim Peters, the core philosophy of python language is as follows:

- Beautiful is better than ugly
- Explicit is better than implicit
- Simple is better than complex
- Complex is better than complicated
- Readability counts

The simplicity and extensibility of python language makes it much popular.

Python uses white space indentation, which can be seen in the program on next chapter. Therefore, the program's structure is easily obtained on the appearance of the program itself.

Programs written in python languages can be saved on the raspberry pi with python file extension '.py'. The designer can execute the programs with or without any graphical output. In this project, monitor and input devices such as keyboard and mouse are not connected, but the designated programs run with respect to the signal at GPIO pins.

5.3 Installation of Packages and Modules

Certain modules are needed to be installed in the Pi, which are not coming in Raspbian OS. Before that updating the firmware is needed to work on Pi hassle free. For every installation, good connectivity to internet is essential. For updating the firmware, on 'Terminal' we can run the following commands:

```
$ sudo apt-get update
$ sudo apt-get  dist-upgrade
$ sudorpi-update
```

5.3.1 Installing OpenCV

OpenCV (Open-Source Computer Vision Library) is a cross-platform library which is mainly aimed at computer vision. It has C++, Python and Java interfaces and supports Windows, Linux, Mac OS, iOS and Android. OpenCV was designed for computational efficiency and with a strong focus on real-time applications.

The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D

models of objects, produce 3D point clouds from stereo cameras, stitch images together to produce a high resolution image of an entire scene, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality, etc.

To install OpenCV in Raspberry Pi, we use the 'Terminal'. The steps to install OpenCV is listed and explained below:

I. Expand File System

Expand the Pi's filesystem to include all available space on the micro-SD card:

```
$ sudoraspi-config
```

In advanced options menu, select 'Expand file-system'. After the process we may reboot the Pi. Rebooting can be done using the command line also.

```
$ sudo reboot
```

After rebooting, your file system should have been expanded to include all available space on your micro-SD card. To make more space free, let's uninstall office software and wolfram from the Pi as

```
$ sudo apt-get purge wolfram-engine
$ sudo apt-get purge libreoffice*
$ sudo apt-get clean
$ sudo apt-get autoremove
```

II. Install Dependencies

The first step is to update and upgrade any existing packages:

```
$ sudo apt-get update && sudo apt-get upgrade
```

CMake helps us to configure the OpenCV building process is then installed.

```
$ sudo apt-get install build-essential cmakepkg-config
```

Next, we need to install some image I/O packages that allow us to load various image file formats from disk. Examples of such file formats include JPEG, PNG, TIFF, etc.:

```
$ sudo apt-get install libjpeg-devlibjasper-dev libpng12-dev
```

The OpenCV library comes with a sub-module named highgui which is used to display images to our screen and build basic GUIs. In order to compile the highgui module, we need to install the GTK development library:

```
$ sudo apt-get install libgtk2.0-dev libgtk-3-dev
```

Many operations inside of OpenCV (namely matrix operations) can be optimized further by installing a few extra dependencies:

```
$ sudo apt-get install libatlas-base-devgfortran
```

These optimization libraries are *especially important* for resource constrained devices such as the Raspberry Pi.

Lastly, let's install Python 3 header files so we can compile OpenCV with Python bindings:

```
$sudoapt-getinstall python3-dev
```

III. Downloading the OpenCV Source Coding

Now that we have our dependencies installed, let's grab the 3.3.0 archive of OpenCV from the official OpenCV repository. This version includes the dnn module which we discussed in a previous post where we did Deep Learning with OpenCV.

```
$ cd ~
$ wget -O opencv.zip
https://github.com/Itseez/opencv/archive/3.3.0.zip
$ unzip opencv.zip
```

For the full install of OpenCV, we also need to grab the opencv_contrib repository as well:

```
$ wget -O opencv_contrib.zip
https://github.com/Itseez/opencv_contrib/archive/3.3.0.zip
$ unzip opencv_contrib.zip
```

IV. Installing 'pip'

Before we can start compiling OpenCV on our Raspberry Pi 3, we first need to install 'pip', a Python package manager:

```
$ wget https://bootstrap.pypa.io/get-pip.py
$ sudo python get-pip.py
$ sudo python3 get-pip.py
```

Our only Python dependency is NumPy (Numerical Puthon), a Python package used for numerical processing. This is installed as follows:

```
$ pip3 install numpy
```

V. Compile and Install OpenCV

We can setup our build using CMake:

```
$ cd ~/opencv-3.3.0/
$ mkdir build
$ cd build
$ cmake -D CMAKE_BUILD_TYPE=RELEASE \
    -D CMAKE_INSTALL_PREFIX=/usr/local \
    -D INSTALL_PYTHON_EXAMPLES=ON \
    -D              OPENCV_EXTRA_MODULES_PATH=~/opencv_contrib-
3.3.0/modules \
    -D BUILD_EXAMPLES=ON ..
```

Finally, we are now ready to compile OpenCV:

```
$ make -j4
```

The -j4 command controls the number of cores to leverage when compiling OpenCV 3. The Raspberry Pi 3 has four cores, thus we supply a value of 4 to allow OpenCV to compile faster. Now the compiling of OpenCV has finished.

```
$ make clean

$ make
```

Now we can install OpenCV on the Pi:

```
$ sudo make install

$ sudo ldconfig
```

VI. Finish Installing OpenCV

After running make install, the OpenCV + Python bindings should be installed in/usr/local/lib/python3.4/site-packages.

5.3.2 The OCR – Space API Package

In chapter 3, we came across with the optical character recognition API used in this project: OCR-Space API. The end point of API is "https://api.ocr.space/parse/image." In order to upload the picture to the cloud and receive the parsed text back in this project, there should be a python wrapper program to the API. A developer from Poland, whose name is Jakub Andrzejewski Zaargh has designed a python wrapper for the OCR-Space API. This wrapper program is used to make request to the API, in this project. In the wrapper program, a class is defined as 'Ocrspace', which can be called in the main program to enable the API.

The wrapper program edited with the API key, which is needed to request the OCR processing is given below:

ocrspace.py (program 1)

```
import requests
apikey = '' # Provide OCR Space API Key
def Ocrspace(filename, overlay=False, api_key=apikey, language='eng'):
    """ OCR.space API request with local file.

    :param filename: Your file path & name.
     :param overlay: Is OCR.space overlay required in your response.
    :param api_key: OCR.space API key.
                    Defaults to 'helloworld'.
    :param language: Language code to be used in OCR.
                    List of available language codes can be found on https://ocr.space/OCRAPI
                    Defaults to 'en'.
    :return: Result in JSON format.
    """

    payload = {'isOverlayRequired': False,
               'apikey': api_key,
               'language': language,
```

```
                }
    with open(filename, 'rb') as f:
        r =
requests.post('https://api.ocr.space/parse/image',
                            files={filename: f},
                            data=payload,
                            )
    #return r.content.decode()
    return r.json()
```

5.3.3 Installation of Other Packages

- 'gTTS' (Google Text to Speech) module is a Python interface for Google's Text to Speech API.

  Installation of gTTS module can be easily done by the help of python package manager – 'pip'. The command is to be run on 'Terminal' as follows:

```
$ pip3 install gTTS
```

- 'Picamera' package can be installed directly through the 'Terminal' with running the following command:

```
$ sudo apt-get install python-picamera
$ sudo apt-get install python3-picamera
```

  The first command installs picamera package usable to python 2x and the second command installs the package into python 3x library.

  (Here 'x' denotes python version).

- ‘IMUTILS’ package is installed using the ‘pip’ command. Here the package is installed on the basis of assumption that, OpenCV and NumPy are already installed in the device.

```
$ pip3 install imutils
```

- The project essentially employs local processing of images. To do that Python Image Library (PIL) should be the latest version. ‘Pillow’ is an python image library fork, which makes its use, simple. This package is installed using ‘pip’ manager as follows:

```
$ pip3 install Pillow
```

- In order to interface the keypad through the GPIO pins, we have to program those pins. For that RPi.GPIO python package is needed to be installed.

```
$ sudo pip install RPi.GPIO
```

CHAPTER 6

CORE PROGRAMS AND THE ACTION OF Pi-Eye

The aims of the project are to make a program to run on Raspberry Pi, for taking picture of a normally printed text in a paper or a book, extract the text on it and read it out and to take a general picture of some physical object or persons to label it and read out the caption. The design is done, concerning the visually impaired users. Thus there are two main programs, written in Python language are loaded into the device. The programs are:

I. **"OCRO.py"** – This program is used to detect text and read it out.

II. **"Micro.py"** – This program is used to detect objects in a picture and read out the caption of image.

Both programs are kept on the default working directory of Raspberry Pi, which is /`home/pi/`. The programs are run on terminal by giving the following command:

```
$ python3 OCRO.py
```

```
$ python3 Micro.py
```

As no graphical user interface as well as SSH will be used in the project, as per the design, these programs have to be executed using the signal received at the GPIO pins, where the HIGH / LOW states of the pre assigned GPIO pins are decided by the input key strokes. For running the main programs using the state of GPIO pins, additional programs are used. In this chapter, the 4 programs; two main programs and two sub-program (or we may say, the control programs) are discussed.

6.1 The OCR Program: "OCRO.py"

As earlier discussed, this program is aimed at extracting text and reading it out. The program mainly consists of four stages:

i. Image Capture
ii. Processing of Image
iii. Optical Character Recognition (OCR)
iv. Text to speech output

Firstly, we are discussing the program as parts, with explaining each important step. After the description the whole program is shown again.

```
#importing needed libraries and packages
import os
from PIL import Image
from picamera import PiCamera
from time import sleep
import cv2
import imutils
import numpy as np
from ocrspace import Ocrspace
from gtts import gTTS
```

A module is a file containing Python definitions and statements. Each program on python works by utilizing the libraries and packages in the working environment. We have already installed needed package files, modules and libraries as we seen in chapter 5. Definitions from a module can be imported into other modules or into the program by calling the defined function in a module.

The OS module in Python provides a way of using operating system dependent functionality. The functions that the OS module provides allows you to interface with the underlying operating system that Python is running on be that Windows, Mac or Linux. Here we have seen that, the operating system is Debian based Raspbian operating system, which actually belongs to the Linux distribution.

PIL is the python imaging library, which was installed as 'pillow' using pip3, as explained in chapter 5. PIL is used for importing images to python program and do assigned processes on it. PIL provides reliable processing, analysis, compression and other features, to apply on images.

Next thing, imported in this program is 'picamera'. It is a package, not just a single module. We can give command as 'import picamera' and get everything in the package or import picamera.encoders to just access the encoders. Here we are accessing the PiCamera class, which is defined on the module. The properties of PiCamera class have been already explained in chapter 4.

Raspbian OS is pre- installed with a lot of basic modules and packages. Time is a basic package, which is mainly used to designate durations of certain lines or process. Sleep is a function defined in the 'time'. Sleep is used to assign a delay before running a certain command line, in the program.

The line ‘import cv2’ enables OpenCV, which is already installed to use in Python 3. In our project, OpenCV is used to process the captured image to make it enhanced. Cv2 package is also used to write and read image files, along the program. The action of OpenCV can be seen in upcoming descriptions.

The next module used in this program is imutils. This package includes a series of convenience functions to make basic image processing functions such as translation, rotation, resizing, displaying Matplotlib images, sorting contours, detecting edges, and much easier with OpenCV and both Python 2.7 and Python 3.

NumPy (Numerical Python) module is essential for OpenCV to do operation like rotation, gaussian curve calculations etc. as the scaling, rotation and other image processes are actally mathematical operation involving matrices.

We already went through the OCR – Space API used in this project and have met the python wrapper for accessing the API, in chapter 5. Thus, we are calling the ‘Ocrspace’ class defined on the python wrapper program.

On the next line, we import python wrapper for Google’s text to speech engine API, which is used to convert the extracted results

```
#Capture process
camera = PiCamera()
camera.resolution = (1280,720)
camera.shutter_speed = 1500000
camera.iso = 1000
```

```
16 camera.brightness = 55

17 camera.contrast = 20

18 sleep(2)

19 camera.capture('f.jpg')

20 camera.close()
```

The lines 11 – 20 of the “OCRO.py” program denote the action of camera, including the capture process. In line number 12, the PiCamera function which has already imported is defined as ‘camera’ for simplicity.

Lines 13 – 17 contain commands for adjusting the camera properties. Camera resolution is set as 1280 X 720 (W X H). Picamera supports only larger dimension in the width and smaller dimension in height. But the project is mainly designed to read books. Most books have portrait orientation, other than landscape orientation. So, the design is to capture the picture with camera rotated in 900. The picture is later rotated to make the it straight. The brightness, contrast, shutter speed and ISO of the camera is then adjusted from the default values, which are listed in chapter 4, to capture the image clearly. The values are designated after adjusting each value, after capturing picture, each time, in a normal in-room low-light environment. The value of shutter speed is given as 1.5 seconds, which is the duration of capture process done by the camera sensor.

The sleep command makes a delay of 2 seconds, before executing the capture command. This time delay is deliberately included, concerning possible shake, that would happen while pressing the input switch. The command in 20th line is executed to capture the image, with above defined properties and save the image with the name “f.png”. The next line closes the action of camera.

```
21  #Rotation

22  img = cv2.imread('f.jpg',0)

23  rot = imutils.rotate_bound(img, 270)

24  file = "10.jpg".format(os.getpid())

25  cv2.imwrite(file, rot)
```

Lines 22 - 25 denote the process of rotation of image. The need for this rotation is already discussed in the previous explanation. The image, which was captured, is read using the command in the line number 22. The command used for rotating the image is shown in the 23rd line. The rotation is done with 'imutils' module. While rotation of image in OpenCV and PIL preserve the ratio of dimension, which would definitely crop out the image and loss the text in it, imutils just rotate the image as a whole. The rotation is done in clock wise direction and so, we have typed in the rotation angle as 2700. Lines 24 and 25 do the job of saving the rotated image. The name of rotated image is given as "10.jpg".

```
26  #Image Enhancing

27  rot = cv2.imread('10.jpg',0)

28  blur = cv2.GaussianBlur(rot,(5,5),0)

29  thresh =
    cv2.adaptiveThreshold(blur,300,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,11,2)

30  file = "9.jpg".format(os.getpid())

31  cv2.imwrite(file, thresh)
```

The next section program, lines from 27 to 31 is used to process the rotated image to make it enhanced. The enhancement is done by employing OpenCV.

The rotated image is read from its directory and smoothened firstly. Image blurring is achieved by convolving the image with a low-pass filter kernel. It is useful for removing noise. It actually removes high frequency content (e.g.: noise, edges) from the image resulting in edges being blurred when this is filter is applied. Line 28 denotes this smoothing process, which is actually blurring the image. We are using the Gaussian filtering method here, in which OpenCV uses a Gaussian kernel. It is done with the function, cv2.GaussianBlur(). We should specify the width and height of the kernel which should be positive and odd. We also should specify the standard deviation in the X and Y directions, sigmaX and sigmaY respectively. If only sigmaX is specified, sigmaY is taken as equal to sigmaX. If both are given as zeros, they are calculated from the kernel size. Gaussian filtering is highly effective in removing Gaussian noise from the image. Here we have set these values as optimum.

The noise removed picture is then provided to undergo enhancement. The enhancement is done by thresholding the image. The function used is cv2.threshold. First argument is the source image, which should be a grayscale image. Second argument is the threshold value which is used to classify the pixel values. Third argument is the maxVal which represents the value to be given if pixel value is more than (sometimes less than) the threshold value. OpenCV provides different styles of thresholding. Actually, we undergo two steps of thresholding. The first one is adaptive thresholding. In this, the algorithm calculates the threshold for a small region of the image. So, we get different thresholds for different regions of the same image and it gives us better results for images with varying illumination.

Gaussian thresholding mode is used in this project. Inside the adaptive thresholding, the image is made to go through a simple thresholding also.

The enhanced image is now saved to the system, with name “9.jpg”.

```
#OCR API
b = Ocrspace(filename='9.jpg', language='eng')
a = '"""%s"""'%b
point = a.find("ParsedText")
start = point+12
next = a.find("FileParseExitCode")
stop = next-4
d = a[start:stop]
if len(d)<5:
    next = a.find("OCRExitCode")
    stop = next-40
    d = a[start:stop]
    e = d.replace(r"\r\n", r" ")
    print(e)
else:
    e = d.replace(r"\r\n", r" ")
    print(e)
```

The lines from 33 to 48 is the process of the extracting text from the processed image. Line 33 denotes calling the API function, uploading the image and requesting OCR language as English. The variable 'b' is the response from the API. The response is in JSON (JavaScript Object Notation) format. An example response is shown in the next chapter.

The text has to be extracted from this JSON response. For that logical operations on strings are used. Command in the 33rd line makes the JSON response, a string by adding three pairs of double quotes. The API seems to be varying in the response format. So, the programmer had to make an approximate logic. Usually, the needed text comes after the word "Parsed Text". So, command in line number 35 finds the name "Parsed Text" in the string and the next line assign the value of position of starting of text. Usually, the needed text is finished before the word "File Parse Exit Code". Again, we detect the positions of this word and from that, the possible position of end of our needed text. Thus, we have found the starting position and ending position of the text, that should be extracted. Next step checks if the length of text between these points is enough. This step is added to check whether the response is correct.

Now the extracted text involves some unwanted parts: "\r\n". Next command removes these unwanted symbols and letters. Thus, the text extraction is completed.

```
49 #Text to Speech

50 tex = e

51 tts = gTTS(text=tex, lang='en')

52 tts.save("2.mp3")
```

```
53  os.system('mpg321 2.mp3')
```

Next part of our process is to make speech output from the extracted text. For that, we have already gone through the TTS (Text to Speech) engine used here. Lines from 51 – 53 do the conversation between GTTS API and the next line plays the mp3 audio file received from the API. The audio is played through the 3.5mm output port of the device.

The whole program "OCRO.py" is given as follows:

OCRO.py (Program 2)

```
1   #importing needed libraries and packages
2   import os
3   from PIL import Image
4   from picamera import PiCamera
5   from time import sleep
6   import cv2
7   import imutils
8   import numpy as np
9   from ocrspace import Ocrspace
10  from gtts import gTTS


11  #Capture process
```

```
camera = PiCamera()
camera.resolution = (1280,720)
camera.shutter_speed = 1500000
camera.iso = 1000
camera.brightness = 55
camera.contrast = 20
sleep(2)
camera.capture('f.jpg')
camera.close()


#Rotation
img = cv2.imread('f.jpg',0)
rot = imutils.rotate_bound(img, 270)
file = "10.jpg".format(os.getpid())
cv2.imwrite(file, rot)


#Image Enhancing
rot = cv2.imread('10.jpg',0)
blur = cv2.GaussianBlur(rot,(5,5),0)
```

```
thresh =
cv2.adaptiveThreshold(blur,300,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY,11,2)
file = "9.jpg".format(os.getpid())
cv2.imwrite(file, thresh)



#OCR API
b = Ocrspace(filename='9.jpg', language='eng')
a = '"""%s"""'%b
point = a.find("ParsedText")
start = point+12
next = a.find("FileParseExitCode")
stop = next-4
d = a[start:stop]
if len(d)<5:
    next = a.find("OCRExitCode")
    stop = next-40
    d = a[start:stop]
    e = d.replace(r"\r\n", r" ")
    print(e)
```

```
else:
    e = d.replace(r"\r\n", r" ")
    print(e)


#Text to Speech
tex = e
tts = gTTS(text=tex, lang='en')
tts.save("2.mp3")
os.system('mpg321 2.mp3')
```

6.2 The Object Detection Program: "Micro.py"

The secondary aim of the project is to detect physical objects in a captured picture and label with an appropriate caption, which describe the things or context of the picture. This job is done by Microsoft's cloud vision API. As we have already seen, this API is a paid one and, in this project, we are utilising the trial version of the API. Like OCR – Space API, there is an API subscription key, which is provided by the vendor. The key is valid for one month of use. As this is only an extension of the project, we can tolerate this limitation.

Like we discussed the "OCRO.py" program, here also we go through the explanations of the parts of program.

```
#Importing Libraries and modules
```

```
import requests
from time import sleep
from picamera import PiCamera
import os
from gtts import gTTS
```

The modules or packages needed in this project are imported to the program on the first steps. The requests library will allow you to send HTTP/1.1 requests using Python. With it, you can add content like headers, form data, multipart files, and parameters via simple Python libraries. It also allows you to access the response data of Python in the same way. The HTTP/1.1 protocol is an application-oriented internet protocol.

As here also the camera has to be enabled, picamera module is imported. The 'sleep' function from 'time' library is included concerning the same reason for its inclusion in the "OCRO.py" program. Similarly, the 'os' module is also imported.

To convert the image caption text to speech output, Google's TTS API is also imported.

```
#Capture of the image
camera = PiCamera()
camera.resolution = (1280,720)
sleep(1)
camera.capture('2.jpg')
```

```
camera.close()
```

The camera capture process is well explained in the precious program. Here we have not given any attributes to the properties of camera. Because, in this program, we do use camera to capture images of general objects. The camera resolution is the only thing, specified. The capture process is delayed by 2 seconds. The captured image is saved as "2.jpeg" file in the working directory. After the capturing, the camera is stopped using the close() command.

```
#Working of Cloud Vision API
subscription_key = "" # Provide API Key
#subscription_key = ""
assert subscription_key
vision_base_url = "https://westcentralus.api.cognitive.microsoft.com/vision/v1.0/"
vision_analyze_url = vision_base_url + "analyze"
{"Content-Type": "application/octet-stream"}
image_data = open(r"/home/pi/2.jpg", "rb").read()
headers    = {'Ocp-Apim-Subscription-Key': subscription_key,"Content-Type": "application/octet-stream" }
params     = {'visualFeatures': 'Categories,Description,Color'}
```

```
response   = requests.post(vision_analyze_url, headers=headers, params=params, data=image_data)
analysis      = response.json()
image_caption = analysis["description"]["captions"][0]["text"].capitalize()
print(image_caption)
```

Next part of the program is the working of Microsoft's cloud vision API. We have two subscription keys and only one of it is used. The vision base URL is the endpoint of the API. We call the 'analyze' function in the API class. In the next steps, header files are input, as per the instructions of the API vendor, to access the cloud server and do the job. The parameters those have to be analysed is also specified. In the next step, the image is uploaded with the requests and needed parameters to the endpoint. Here also, the API response is given in JSON (JavaScript Object Notation) format. The needed text in the response is extracted in to the variable 'image_caption'.

```
#TTS
tex = image_caption
tts = gTTS(text=tex, lang='en')
tts.save("2.mp3")
os.system('mpg321 2.mp3')
```

The extracted label of the image is now fed to the Google's TTS engine. The language is specified. The speech file in MPEG Layer 3 (mp3) format is saved on the same directory of the program. Then the audio is fed to the 3.5 mm output.

The whole program "Micro.py" is given as follows:

Micro.py (Program 3)

```
#Importing Libraries and modules
import requests
from time import sleep
from picamera import PiCamera
import os
from gtts import gTTS


#Capture of the image
camera = PiCamera()
camera.resolution = (1280,720)
sleep(1)
camera.capture('2.jpg')
camera.close()


#Working of Cloud Vision API
```

```
subscription_key = "" # Provide Key
#subscription_key = ""
assert subscription_key
vision_base_url = "https://westcentralus.api.cognitive.microsoft.com/vision/v1.0/"
vision_analyze_url = vision_base_url + "analyze"
{"Content-Type": "application/octet-stream"}
image_data = open(r"/home/pi/2.jpg", "rb").read()
headers    = {'Ocp-Apim-Subscription-Key': subscription_key,"Content-Type": "application/octet-stream"}
params     = {'visualFeatures': 'Categories,Description,Color'}
response   = requests.post(vision_analyze_url, headers=headers, params=params, data=image_data)
analysis      = response.json()
image_caption = analysis["description"]["captions"][0]["text"].capitalize()
print(image_caption)


#TTS
```

```
tex = image_caption
tts = gTTS(text=tex, lang='en')
tts.save("2.mp3")
os.system('mpg321 2.mp3')
```

6.3 Controlling Programs

As the above programs have to be worked as per the wish of the user, we have to program the GPIO pins, to which the keypad is connected. Generally programming the GPIO in Python script involves following steps:

1. Import the RPi.GPIO package.
2. Set the numbering style to be used. We use the method GPIO.setmode() for this. It takes either GPIO.BOARD or GPIO.BCM as the parameter. GPIO.BOARD stands for physical numbering and GPIO.BCM stands for Broadcom numbering. In order to keep things simple, we will use only GPIO.BOARD here.
3. Set up the necessary input and output pins.
4. Read inputs and give outputs.
5. Clean up GPIO and exit.

For programming the GPIO pins to work as input pins, any 3.3v pin or Ground pin should be used as reference pin. Otherwise, it would make an invalid floating condition. That should be avoided by using a 3v3 pin or a ground pin and an in-built pull up or pull-down resistor.

A pin can be set as input as follows:

```
GPIO.setup(channel, GPIO.IN, PUD)
```

Here, the channel is the pin number. PUD can be either GPIO.PUD_DOWN or GPIO.PUD_UP. It tells you whether to use the inbuilt pull up or the pull-down resistor. If the 3v3 pin is used, we have to use GPIO.PUD_DOWN (pull down resistor), and for the ground pin, we need to use GPIO.PUD_UP (pull up resistor). The choice of 3v3 or ground is up to you. The only difference is the value read from an open circuit and closed circuit. For the 3v3 pin (GPIO.PUD_DOWN), the input value of the open circuit will be 0 and of a closed circuit will be 1. For a ground pin (GPIO.PUD_UP), open circuit is 1 and closed circuit is 0.

In this project we are using the board numbering. The common wire of the keypad (connector number 5) is connected to the 39th (physical pin number), which is a ground pin. The key 4 is connected to pin number 37 (Broadcom Number – GPIO 26), key 3 to pin number 35 (BCM Number – GPIO 19), key 2 to pin number 33 (BCM Number – GPIO 13) and key 1 is connected to pin number 31 (BCM Number – GPIO 6).

As the common connection is made to ground pin, the programs have to be designed using the logic GPIO.PUD_UP. That is the designated program to that pin should run, when the pin reads 0. The pin is made 0, only when the corresponding key is pressed.

The program to run “OCRO.py” when key 3 is pressed is given below:

3ocrt.py (Program 4)

```
#Importing needed packages and libraries

import RPi.GPIO as GPIO
```

```
import time
import os


#Setting the GPIO Numbering and Logic
GPIO.setmode(GPIO.BOARD)
GPIO.setup(35, GPIO.IN, GPIO.PUD_UP)


#Running selected program
while True:
    input_state = GPIO.input(35)
    if input_state == False:
        print('Button 3 is Pressed')
        os.system('/home/pi/OCRO.py')
        time.sleep(0.2)
```

The program to run “Micro.py” when key 4 is pressed is given below:

4micro.py (Program 5)

```
#Importing needed packages and libraries
import RPi.GPIO as GPIO
```

```
import time
import os


#Setting the GPIO Numbering and Logic
GPIO.setmode(GPIO.BOARD)
GPIO.setup(37, GPIO.IN, GPIO.PUD_UP)


#Running selected program
while True:
    input_state = GPIO.input(37)
    if input_state == False:
        print('Button 4 is Pressed')
        os.system('/home/pi/Micro.py')
        time.sleep(0.2)
```

The other two keys of the keypad is not used yet. These can be utilised in further development on the project.

6.4 Action of the Device

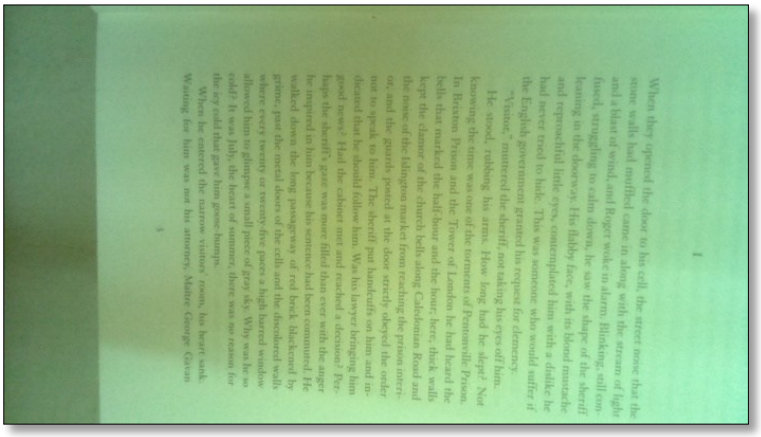
The device is programmed and packaged in a suitable case. Now it is the time of employing the device. The power supply is given from the power bank. The device will be ready in about 1 minute, which is the booting time. The headphone is plugged into the 3.5mm port correctly and it is put on over ears. A Wi-Fi internet connection, which has already paired with the device, should be there.

Now to run the "OCRO.py" program, we have set the text book or paper, which carries the text to be read. Assuming the environmental lighting to be optimum, the device is placed on the page, with the edges of the device almost parallel to the page and camera part (near the top edge of device) is kept at nearly the centre of the page. Note that, on the upper side of the device there is a swollen arrow marking. This marking is drawn to help the visually impaired user to assure that device is not inverted. Now the device is elevated vertically up to a height of ~25 cm, above the plane of paper. This distance was noted by trial-and-error process. Now the keypad number 3, which have 3D marking of a ring is pressed to run the program. The device has to be made stationary at least 5 seconds in order to capture fine image and therefore getting good OCR. Now the TTS audio is played over the headphones.

For executing the "Micro.py" program, the device is placed in a position in the user's hands, with camera directing towards the object. Now the key number 4 is pressed and we can hear the image caption, within 1 or 2 minutes, according to the speed of internet connectivity.

Note that, the above instructions are given, considering the visually impaired user.

An example image which was captured during the execution of “OCRO.py” is given below. Alongside, the rotated as well as the enhanced images are given.



1

When they opened the door to his cell, the street noise that the stone walls had muffled came in along with the stream of light and a blast of wind, and Roger woke in alarm. Blinking, still confused, struggling to calm down, he saw the shape of the sheriff leaning in the doorway. His flabby face, with its blond mustache and reproachful little eyes, contemplated him with a dislike he had never tried to hide. This was someone who would suffer if the English government granted his request for clemency.

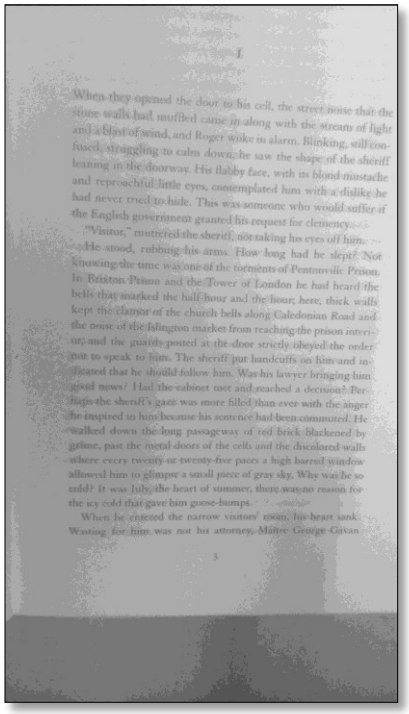
“Visitor,” muttered the sheriff, not taking his eyes off him.

He stood, rubbing his arms. How long had he slept? Not knowing the time was one of the torments of Pentonville Prison. In Brixton Prison and the Tower of London he had heard the bells that marked the half-hour and the hour; here, thick walls kept the clamor of the church bells along Caledonian Road and the noise of the Islington market from reaching the prison interior, and the guards posted at the door strictly obeyed the order not to speak to him. The sheriff put handcuffs on him and indicated that he should follow him. Was his lawyer bringing him good news? Had the cabinet met and reached a decision? Perhaps the sheriff’s gaze was more filled than ever with the anger he inspired in him because his sentence had been commuted. He walked down the long passageway of red brick blackened by grime, past the metal doors of the cells and the discolored walls where every twenty or twenty-five paces a high barred window allowed him to glimpse a small piece of gray sky. Why was he so cold? It was July, the heart of summer, there was no reason for the icy cold that gave him goose-bumps.

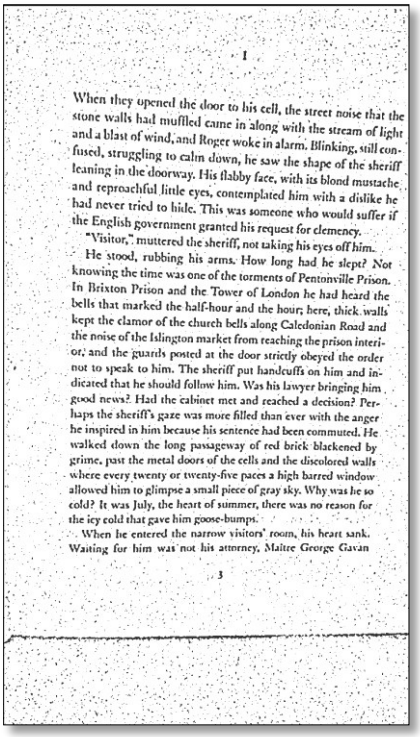
When he entered the narrow visitors’ room, his heart sank. Waiting for him was not his attorney, Maître George Gavan

3

“f.jpeg” (Captured Image) Fig. 8



“10.jpeg" (Rotated Image) Fig. 9



1

When they opened the door to his cell, the street noise that the stone walls had muffled came in along with the stream of light and a blast of wind, and Roger woke in alarm. Blinking, still confused, struggling to calm down, he saw the shape of the sheriff leaning in the doorway. His flabby face, with its blond mustache and reproachful little eyes, contemplated him with a dislike he had never tried to hide. This was someone who would suffer if the English government granted his request for clemency.

“Visitor,” muttered the sheriff, not taking his eyes off him.

He stood, rubbing his arms. How long had he slept? Not knowing the time was one of the torments of Pentonville Prison. In Brixton Prison and the Tower of London he had heard the bells that marked the half-hour and the hour; here, thick walls kept the clamor of the church bells along Caledonian Road and the noise of the Islington market from reaching the prison interior, and the guards posted at the door strictly obeyed the order not to speak to him. The sheriff put handcuffs on him and indicated that he should follow him. Was his lawyer bringing him good news? Had the cabinet met and reached a decision? Perhaps the sheriff’s gaze was more filled than ever with the anger he inspired in him because his sentence had been commuted. He walked down the long passageway of red brick blackened by grime, past the metal doors of the cells and the discolored walls where every twenty or twenty-five paces a high barred window allowed him to glimpse a small piece of gray sky. Why was he so cold? It was July, the heart of summer, there was no reason for the icy cold that gave him goose-bumps.

When he entered the narrow visitors’ room, his heart sank. Waiting for him was not his attorney, Maître George Gavan

3

"9.jpeg" (Enhanced Image) Fig. 10

CONCLUSION

The aim of the project, to make a text in a book or a paper reader and a detector which analyse objects before the user and gives an idea about it as audio is successfully achieved. The programming is done on the credit card sized computer, Raspberry Pi, using Python programming language. The device is set up with camera, power supply and an earphone and provided Wi-Fi internet connectivity. The working of the device is verified. Speech output of the text in the given book/page or the caption of an image in front of the user is successfully given out.

In this project, our idea was to make a low-cost text book reader for visually impaired persons; especially students. Devices and gadgets which are already in the market are very expensive. Starting prices of these are nearly ₹20,000, which is not a possible price tag for proletarians. In this project, the overall cost is below ₹4,000 only. If product like this is made in an industry, the expense will surely come down, with purpose-oriented computer chips.

We hope that, this project would make visually impaired people's life, brighter. As they can read books or have a general idea of the environment without a direct involvement of a helper.

Further Developments

- The power supply can be made integrated.
- The device can be made much handier with a well-designed case.
- Camera can be improved to the version 2, which is already released.
- A low powered speaker may be integrated in the device.
- Free APIs may come in future which can make the things more beautiful.
- The computer board can be designed as purpose oriented, which could make the action powerful, smaller as well as less expensive.

References

1. "Milestones: First Semiconductor Integrated Circuit (IC), 1958". IEEE Global History Network. IEEE. Retrieved 3 August 2011.
2. Microprocessor Theory and Applications with 68000/68020 and Pentium by M. Rafiquzzaman.
3. Morgan, Jacob. (2014, May 13). Simple-explanation-internet-things-that-anyone-can-understand. Retrieved from: https://www.forbes.com/.
4. JRank Articles. Electronics - History - Radio, Tube, Television, and Integrated Circuits. Retrieved from: http://science.jrank.org/
5. Farida. (2008, February 10). SSI, MSI, LSI [Web log post]. Retrieved from: http://integratedcircuithistory.blogspot.in/
6. Bhuyan, Muhibul. (2016). History of Electronics. 10.13140/RG.2.1.4683.4967.
7. Philip, Nooble. (2017, July 11). How to begin with Raspberry Pi GPIO programming using Python. Retrieved from: https://opensourceforu.com/
8. Peters, Tim (19 August 2004). "PEP 20 – The Zen of Python". Python Enhancement Proposals. Python Software Foundation.
9. Rosebrock, Adrian. (2016, April 18). Install guide: OpenCV3. Retrieved from https://www.pyimagesearch.com
10. Raspberry Pi 3 B Pin Out Diagram. (2017, December 7). Retrieved from https://www.element14.com
11. https://elinux.org/RPi_Hub
12. https://opensource.com/resources/raspberry-pi